test_code_handbook_2026-10-01

目录

- SpecDec 项目：测试与诊断代码手册
- 0. 这份手册怎么用
- 1. 当前总状态（跟踪用）
- 2. 测试体系分层：谁测什么
- 3. 全量 CPU 自测矩阵（**本手册实测：788 PASS / 0 FAIL**）
- 4. 对齐门（引擎 ↔ HF）
- 5. 缓存 / 分块 / 位置等价门
- 6. 里程碑门 G1–G5（历史工具，按 docstring 归类）
- 7. 分歧定位诊断（主线，按"问题"组织）
- 8. 治理与记录
- 9. 方法论：这些工具共同的实现方法
- 10. 命令速查
- 11. 附录 B：文档与产物索引（本阶段）
- 附录 A：`specdec/engine` 全量代码清单（自动扫描）

SpecDec 项目：测试与诊断代码手册

版本：2026-10-01 · 提交 `f6335f5` · 引擎目录 `specdec/engine`（139 个 `.py`）

用途：后期跟踪。每个工具一行到一段，写清 **作用 / 实现方法 / 入口 / 产物 / 状态。**

边界：本手册是**工具索引与实现说明**，不是结论文档。结论以 `docs/` 下的报告与台账为准；

本手册**不**放宽任何门槛、**不**替代任何验收记录。

---

0. 这份手册怎么用

0.1 重新生成 PDF（改完 `.md` 跑一条命令）

```
cd D:\RealDSH\specdec
.\.venv\Scripts\python.exe -X utf8 scripts\md_to_pdf.py docs\test_code_handbook_2026-10-01.md --inventory
```

- 源文件是 Markdown（好改、好 diff）；PDF 只是产物。
- `--inventory` 会在末尾**实时扫描** `specdec/engine/*.py` 追加"全量代码清单"⇒ **新增文件会自动出现在下一次 PDF 里。**
- 渲染链路：Markdown → 自写极简渲染器 → HTML + 打印 CSS → 本机 **Chrome/Edge** `--headless --print-to-pdf`。

不需要安装任何依赖（中文用系统字体 Microsoft YaHei）。实现见 `scripts/md_to_pdf.py`。

0.2 状态标签（全仓统一口径）

标签	含义
已验证通过	有落盘产物 + 可复核的判据，且本轮/该轮实际跑过
已实现未验证	代码在、能跑，但判据未跑或未通过
设计完成未实现	只写了方案，未执行
已撤回 / 判 INVALID	曾经的结论被推翻（**原文保留 + 标记**，不删除）
默认关闭	诊断开关，不设环境变量时**行为与原来逐字节一致**

0.3 三条最常见的误用（先看这里）

1. **每次 `pwsh`/命令行调用都是新进程**：环境变量不会保留。设置变量与执行命令**必须写在同一行**
（本项目因此浪费过 1 次 GPU 运行）。
1. **摘要一致 ≠ 逐位一致**：`sha16` 只能叫"摘要一致"；要逐位就必须 `torch.equal`（逐张量/逐元素）。

2. **后续 token 相同 ≠ 缓存状态相同**：两臂可以生成完全相同的 token 序列而 KV 缓存已经不同（本项目已实测到）。

---

1. 当前总状态（跟踪用）

1.1 协议与门槛

项	状态
冻结提案	`docs/proposal_freeze_2026-10-01.md`（**6 主条件**：BF16 KV × {2K,4K,8K} × {AR,Spec} + **2 补充**：8K V-only FP8）
正式正确性门 **B6**	`SpecDec ids == target-only greedy ids`（精确相等）+ 全有限 + 多次确定性 —— **未通过**
因此禁止	acceptance 测量 / pilot / final；正式矩阵；FP8 主线（BF16 主线未结）
生产默认	`model_self.Build_Rope_cache` 仍是**设备上**算 `inv_freq`（未采纳对齐）；权重/精度/后端/proposal 均未改
资源护栏	`g5_8k_spec_variance_probe.commit_guard()`（安全线 16 GiB）+ P-state 预检 + 双模型降峰值加载

1.2 已证实的结论（可直接引用）

#	结论	关键证据		
1	**权重映射与配置无误**	338/338 张量逐位一致；checkpoint 真带 84 个 q/k/v bias（338 = 254 + 84）、无 `lm_head.weight`；v5 硬编码 `bias=True`		
2	**HF 缓存克隆缺陷（已修）**	旧 `xb_probe` 克隆只填 keys/values ⇒ `is_initialized=False`、`get_seq_length()==0`、首次 `update()` 清空历史；CPU 实测 6-token 克隆后追加 1 token 长度=1		
3	**对齐 RoPE 频率来源后，C1 与 HF 逐位一致**	56/56 K/V 张量逐位相同 + 末位 151936 维 logits 逐位相同；`inv_freq` 仅在 GPU/CPU 两个下标差 1 个 fp32 ULP		
4	**放大机制**	L0 的 60 个元素差异 → L1 52980 → L2 1705648 → … → L27 3374171（1 ULP 种子被放大 ~5 个数量级）		
5	**"单步 vs 块"数值差是共性**	HF 自己的 s=1 与 s=4 在四行上数值不同（max\	Δ\	0.17–0.25）而 argmax 全同
6	**AR/Spec 首个分歧（index 47 / 位置 8239）的成因 = 缓存历史**	同历史换形态 argmax 不变；同形态换历史 argmax 翻转；两历史 55/56 张量逐位不同、最早分叉在位置 8192（28/28 层）		
7	**本决策点无逻辑错误可检出**	8/8 数据流核对（行↔位置↔token 三向一致）+ mask 泄漏检查（含阳性对照）+ rollback 边界一致		
8	**UCG（统一执行几何）原型被否定**	F13 index 47：AR 6702 / Spec 6652		

1.3 已撤回 / 判 INVALID（不要引用）

原说法	状态
阶段 B 跨实现对照"12.54–16.49"及其派生表述	**INVALID**（HF 侧克隆空历史）
"上一轮 HF 探针 2096 末块 ⇒ prefill 分块日程敏感性"	**撤回**（旧探针日程与新协议**相同**：4×2048 + 末块 48）
"L0 V 逐位相等 ⇒ 输入 hidden 与投影都可逐位复现"	**过强**（只直接证明 v_proj 输出相等）
"RMSNorm 按执行顺序排除"	**错**（`input_layernorm` 在 K/V 投影**之前**）
"CPU 上表逐位相同 ⇒ 排除 RoPE"；"位置集合重合 ⇒ 确认因果"	**过强**（前者不能排除设备路径差异，后者只是支持性）
"由 60/2M 反推扰动 ≈0.2 fp32 ULP ⇒ 排除 TF32 ⇒ 限 fp32 最后一位"	**整条撤回**（无依据推断）

原说法	状态				
"`max\	Δ\	=0.015625` 是 1 ULP"	**错**（该处 bf16 间距 0.0078125 ⇒ 2 个间距；实测 \	Δ\	/间距 ∈ {1,2,4,8,16,64}）

1.4 尚未验证（不要当成已通过）

1. **B6 本身**：对齐 RoPE 后是否仍失败 —— 未跑完整 B6；已知真实 AR/Spec 仍不一致（1 个 token）。
2. **其它 family / 长度 / chunk 日程 / backend**：逐位一致的结论只覆盖 F13 / 8192 / chunk=2048 / eager / bf16。
3. **C2 与两臂**（对齐后）：未测。
4. **fp32 真实间隔**：`classify_bf16_compare` 需要 `fp32_margin`（只能由 fp32 运行得到）⇒ 相关工具**未调用**、

不给出任何 `BF16_NEAR_TIE` 判定。

1. **生产是否采纳频率来源对齐**：未决定（它**既非必要也非充分**，且会重新基线化全部历史数值）。
2. **历史结论审计**：F13/index50 取证、四格、UCG 否定、归约开关实验都跑在旧数值谱系上，观测成立、**归因**待重看。

1.5 数值谱系（lineage）：跨谱系不得直接比较

谱系	RoPE `inv_freq` 来源	产物
A（历史）	target 在 **device** 上算（原实现）	2026-09-29 ~ 2026-10-01 早期全部运行（含 F13 capture、UCG、四格）
B（本轮）	target 用 **CPU-FP32** 算好再搬运（`SMOKE_TARGET_ROPE=cpu_fp32`）	`c1rope_*`、`divasc_*`、`divcap_*`、以及对齐后的 `smoke8k_eager_*`

对齐使 8345 万个 K/V 元素与 139637 个 logits 维度改变 ⇒ A/B 数值**不可混比**，历史产物**原样保留**。

---

2. 测试体系分层：谁测什么

层	目的	代表工具	需 GPU	典型产物
L0 纯逻辑自测	不加载模型，验证判定/索引/对齐/克隆/数值约定	19 个 `*_selftest_cpu.py` + 内置自测	✗	stdout（PASS/FAIL），个别写 JSON
L1 引擎↔HF 对齐门	从零实现的引擎是否与 HF 数值一致	`align_ours`、`alignment`、`gpu_bf16_align_probe`、`c1_rope_source_diag`	✓	对齐报告 JSON / 快照
L2 缓存与分块等价门	cached decode == full forward; chunked prefill == single-shot	`test_cache_equiv`、`test_chunked_prefill`、`test_chunked_fp32`、`align_cache_*`	✓	等价性报告
G1–G2 目标与可行性	1.5B target 正确性；8K 可行性/峰值	`g1_target_15b`、`g2_feasibility`、`g2_operational`、`g2_peak_probe`	✓	门报告 JSON
G3 FP8 KV	量化往返/短窗/消融/质量门	`fp8_vonly_v1_roundtrip`、`fp8_kv_v3_chunked_ablation`、`fp8_kv_v4_dppl` …	✓	质量/消融 JSON
G4 SpecDec 组件与流程	draft 正确性、cache append/trim/rollback、EOS 分支、主循环不变量、**B6 正确性门**	`g4_0_*`、`g4_b4_cache_tests`、`g4_b5_*`、`g4_b6_correctness`、`g4_specdec_index_tests`	部分	门报告
G5 8K 运行与测量	单次 smoke / 稳定化 / 峰值 / 成本分解 / 主张审计	`g5_8k_smoke_cudnn`、`g5_stability_probe`、`g5_cost_breakdown`、`g5_attn_claims_audit`	✓	运行 JSON + `mem_timeline`
分歧定位诊断	定位"AR/Spec 不一致"与"引擎↔HF 不一致"的成因	`xb_probe`、`hf_serial_block_probe`、`l0_k_replay`、`c1_rope_source_diag`、`f13_i50_cells`、`ar_spec_consistency_cpu`	部分	快照 PT + 比较 JSON
治理与记录	family 独立性、schema、状态字段、判分口径	`verify_family_independence_cpu`、`g5_schema_selftest_cpu`、`results_schema`、`g5_driver`	✗	守卫 JSON

---

3. 全量 CPU 自测矩阵（本手册实测：788 PASS / 0 FAIL）

一次性跑全部（PowerShell，逐模块独立进程）：

```
cd D:\RealDSH\specdec
$env:PYTHONIOENCODING="utf-8"
$mods = @("cache_clone","decision_rows_selftest_cpu","sb_align_selftest_cpu",
 "dispatch_judge_selftest_cpu","unified_geometry_selftest_cpu","g5_schema_selftest_cpu",
 "verify_readiness_selftest_cpu","prefill_finite_selftest_cpu","qk_scope_selftest_cpu",
 "smoke_checks_selftest_cpu","smoke_midstep_selftest_cpu","spec_verify_index_selftest_cpu",
 "attn_phase_selftest_cpu","g4_part4_selftest_cpu","g5_8k_divergence_2x2_selftest_cpu",
 "diag_fields_selftest_cpu","attn_l27_g012_selftest_cpu","attn_l27_tail_ab_selftest_cpu",
 "verify_family_independence_cpu")
foreach ($m in $mods) {
  $out = & .\.venv\Scripts\python.exe -X utf8 -m "specdec.engine.$m" 2>&1
  "{0,-40} rc={1} {2}" -f $m, $LASTEXITCODE, (($out | Select-String "PASS \d+ / FAIL \d+" | Select-Object -Last
1).Line)
}
```

模块	PASS	FAIL	测什么（实现方法一句话）
`g5_schema_selftest_cpu`	**162**	0	结果 schema/判定分类器/元数据（含 bf16 近贴边四条件门槛）
`g4_part4_selftest_cpu`	**68**	0	Part4 测量工具链（不加载模型）
`verify_readiness_selftest_cpu`	**54**	0	G1–G5 与**五类状态**的回归
`attn_phase_selftest_cpu`	**43**	0	prefill-only 融合注意力分支（stub 后端）
`sb_align_selftest_cpu`	**38**	0	S/B 对照的**行对齐/可见范围**逻辑
`prefill_finite_selftest_cpu`	**37**	0	`prefill_finiteness` 契约 v2
`qk_scope_selftest_cpu`	**36**	0	QK-FP32 干预的**范围与生效性**（假阳性防护）
`diag_fields_selftest_cpu`	**35**	0	`diag_fields` 轻量回归
`attn_l27_g012_selftest_cpu`	**33**	0	L27 G0/G1/G2 对照的运行前自测
`decision_rows_selftest_cpu`	**28**	0	决策行/槽位/因果可见范围/独立存储
`smoke_midstep_selftest_cpu`	**28**	0	`_propose` **中间步**有限性（stub 模型，同一套 hook 代码）
`smoke_checks_selftest_cpu`	**27**	0	8K smoke 的检查逻辑
`unified_geometry_selftest_cpu`	**26**	0	UCG 原型几何（GPU 前必须过）
`verify_family_independence_cpu`	**25**	0	**治理守卫**（见 §8）
`cache_clone`（内置）	**24**	0	克隆 8 项不变量 + 旧实现回归（必须抓得住错法）
`attn_l27_tail_ab_selftest_cpu`	**22**	0	末端传播 A/B 的运行前自测
`dispatch_judge_selftest_cpu`	**22**	0	按 phase 的后端分流判定
`g5_8k_divergence_2x2_selftest_cpu`	**22**	0	2×2 定点诊断纯逻辑
`spec_verify_index_selftest_cpu`	**16**	0	`_verify` 的索引/位置/mask/cache 语义哨兵
小计（19 模块）	**746**	**0**	—

内置自测（在各工具自己的开关下）

工具	开关	PASS/FAIL	测什么
`target_rope_scope`	`python -m specdec.engine.target_rope_scope`	**14 / 0**	实例级作用域：target 全 `cpu_fp32`、draft 全 `original` 且数值不变、未知实例抛错、异常后还原
`f13_i50_cells`	`CELLS_SELFTEST=1`	**9 / 0**	完整快照/独立克隆/格内复现/mask 泄漏检查（含阳性对照）
`xb_probe`	`XB_SELFTEST=1`	**11 / 0**	权重映射（bias 映射而非删除）/数值只记录/缓存克隆
`hf_serial_block_probe`	`HFSB_MODE=selftest`	**8 / 0**	短输入下的行对齐与实际 mask 支撑集

注入故障回归（"检查必须拦得住"）：`g5_8k_smoke_cudnn` 的 `SMOKE_SELFTEST` 四个场景

（`nonfinite` / `missing` / `invariantfail` / `dispatchfail`）**全部按设计判否、退出码 5、且落盘**——这是**负向对照**，证明门不是"恒真"。

合计：19 模块 746 + 4 内置自测 42 = 788 PASS / 0 FAIL。

---

4. 对齐门（引擎 ↔ HF）

工具	作用	实现方法	入口	产物	状态				
`align_ours.py`	**L1 门**：自研前向 vs HF（同 ids）	从 config 建引擎、逐张量拷贝权重（strict）、两边各跑若干 prompt，比 max\	Δ\	与 top1	`python -m specdec.engine.align_ours --model ... --tol 1e-3`	stdout	早期门，**未在本手册重跑**		
`alignment.py`	A4 门（另一套口径）	同上思路的独立实现	`python -m …`	stdout	历史门				
`gpu_bf16_align_probe.py`	CUDA bf16 中等长度的 OURS vs HF 对齐	GPU bf16 前向对照	无 env	stdout	历史探针				
`gpu_fp16_align_probe.py`	GPU fp16 对齐（fp32 累加路径）	同上，fp16	无 env	stdout	历史探针				
`cpu_nll_sanity.py`	CPU fp32 的 NLL/argmax 健全性	引擎 vs HF 在 CPU fp32 上比 NLL	无 env	stdout	历史门				
`c1_rope_source_diag.py`	**本轮核心**：只把 `inv_freq` 换成 CPU-FP32 后的 C1 诊断	① `gate`：**不加载模型**，用真实 config 参数在 CPU 算 `inv_freq`，与已记录的 HF `inv_freq` 逐位比、四个 chunk 的 cos/sin 与已捕获 HF 表逐位比；② `run`：替换 `model_self.Build_Rope_cache`（**逐行复刻**，只改第一行），跑一次完整 prefill 到 8192，落盘 logits + 28 层 K/V；③ `compare`：三方只读比较	`L0D_MODE=gate\	run\	compare`	`c1rope_gate_*.json`、`c1rope_run_*.json/.pt`、`c1rope_compare_*.json`	**已验证通过**（56/56 + logits 逐位相同）		
`l0_k_diff_cpu.py`	从既有 C1 快照列出第 0 层 K/V 的**全部差异清单**	固定路径读取；断言存的是"精确 bf16 值"；按 dtype 实测间距（位模式 +1）算 ULP 比；输出 60 行清单与结构汇总	无 env（零 GPU）	`l0k_diff_*.json`	**已验证通过**（60 项，跑两次一致）				
`l0_k_replay.py`	第 0 层 K 生成的**最小重放**：节点对照 + 复现门 + 条件式 RoPE 表替换	只跑 `Embedding → input_layernorm → k/v_proj → RoPE`（不跑其余 27 层/attention/LM head）；**按原 chunk 形状**（2048 行）计算后再抽取；先过复现门（重放 `k_post` 必须与快照逐位相同）再归因；`subst` 只在门过且 `k_pre` 两侧逐位相同时才允许	`L0_MODE=ours\	hf\	compare\	subst\	table`	`l0_replay_*.json` + `dump.pt`	**已验证通过**（复现门两侧全过；只换 cos/sin 逐元素解释 60 个差异）

工具	作用	实现方法	入口	产物	状态			
`target_rope_scope.py`	把 CPU-FP32 频率来源**只绑定到 target 实例**	包装**类级** `QwenLM._run` 记录"当前实例"→ 包装模块级 `Build_Rope_cache` 按 当前实例 `is target` 分派；**实例未知即抛错**（绝不静默用错表）；`restore()` 用 `is` 核查还原	被 `g5_8k_smoke_cudnn` / `f13_i50_cells` 调用	作用域元数据 + 调用记录 + 恢复核查	**已验证通过**（自测 14/0）			
`xb_probe.py`	自研 vs HF 的**同路径**对齐探针（协议 S*）	两侧**各自独立**构建缓存；**阶段闸门** `XB_STAGE`（默认只到 C1）；保存克隆不变量断言	`XB_IMPL=hf\`	ours、`XB_STAGE=c1\`	c2\	ab、`XB_SELFTEST=1`	`xb_20260101_{ours,hf}_*.{json,pt}`	C1/C2 **VALID**；阶段 B **INVALID**
`xb_compare_cpu.py`	两侧产物的**只读**比较（阶段感知）	只比较两边都执行过的阶段；未执行记 `executed=False` 且**不当成相同**；协议/精度逐项断言；输出名带写入时间戳	无 env（零 GPU）	`xb_20261001_compare_*.json`	已验证（含"阶段 B 拒绝比较"）			
`hf_serial_block_probe.py`	**HF 侧**单步/块对照（外部实现）	两臂从**共享同一初始缓存**的深拷贝出发；行对齐 + **实际加法掩码**支撑集检查（`output_attentions=True`）	`HFSB_MODE=selftest\`	run、HFSB_RED_REDUCTION、HFSB_FAMILY、HFSB_PRE`	`hfsb_20261001_*.json`	克隆修正后仍**不受 bug 影响**；行 `[16,20,11,220]`		
`xb_hfsb_row_recompute_cpu.py`	用**既有产物**重算"同协议跨实现行对照"	三重前置校验（协议/精度/**历史位级**：两支独立 HF 运行的第 0 层 K sha 相同）→ 只读比较四行 argmax 与行 sha	无 env（零 GPU）	`xb_hfsb_row_recompute_*.json`	已验证（四行 argmax 全同、行级逐位 0/4）			

5. 缓存 / 分块 / 位置等价门

工具	作用	实现方法	入口
`test_cache_equiv.py`	L2 门：cached decode == full forward	同一批 token 分别走"整段前向"与"缓存逐步"，比 logits	无 env
`test_chunked_prefill.py`	chunked prefill == single-shot prefill	分块与整段各跑一次，比 cache 与 logits	无 env
`test_chunked_fp32.py`	隔离"0.55 cache 差距是否 fp16 舍入"	换 fp32 复跑同一对照	无 env
`align_cache_diag.py` / `align_cache_real.py`	全前向与缓存解码**在哪里**分叉（含真实权重）	逐层/逐位置对照	无 env
`spec_verify_index_selftest_cpu.py`	`_verify` 的**索引/位置/mask/cache 语义**哨兵	CPU 哨兵；含"传入 cache 被就地改写 ⇒ 两条分支必须各自 clone"	`python -m …`

工具	作用	实现方法	入口
`decision_rows.py`（+ `decision_rows_selftest_cpu.py`）	决策行/槽位表/因果可见范围/**独立存储**断言	`slot_table(pre,kp)` 给出每槽的来源、预测位置、可见 key；`require_*` 系列	库 + 自测
`sb_align.py`（+ `sb_align_selftest_cpu.py`）	溯源保持的**行对齐**与跨进程可比摘要	`stable_sha`（内容 sha256）；`require_full_cache`（形状/长度/dtype/finite 硬检查）；`cache_digest_diff`	库 + 自测
`cache_clone.py`	**可靠克隆** + 8 项不变量 + 旧实现回归	`clone_cache_full/clone_list_cache_full` = `copy.deepcopy`；`require_clone_invariants` 断言 K/V 位级、长度、初始化状态、无共享存储、追加后历史保留、兄弟副本不受影响	`python -m specdec.engine.cache_clone`
`diag_cache_cmp.py` / `diag_checks.py`	缓存**真正独立**快照与可检测比较；诊断校验原语	纯函数 + 可检测（不只是"打印"）	内置自测

6. 里程碑门 G1–G5（历史工具，按 docstring 归类）

门	工具	作用
G1	`g1_target_15b.py`	1.5B target 正确性门
G2	`g2_feasibility.py`、`g2_operational.py`、`g2_peak_probe.py`、`probe_gpu.py`、`g5_chunk_fit_sweep.py`	8K 可行性/正式门/峰值方差/chunk 分档拟合
G3	`fp8_vonly_v1_roundtrip.py`、`fp8_kv_v1_roundtrip.py`、`fp8_kv_v2_short_window.py`、`fp8_kv_v3_chunked_ablation.py`、`fp8_kv_v3_localize.py`、`fp8_kv_v4_dppl.py`、`fp8_kv_v4_probe.py`、`fp8_kv_group_probe.py`、`fp8_scale_diag.py`、`fp8_vonly_8k_operational.py`	FP8 KV 往返/短窗/消融/分叉定位/质量门(V4 PPL)/分组 scale 归因
G4	`g4_0_draft_correctness.py`、`g4_0_drift_diag.py`、`g4_b3_dual_model_feasibility.py`、`g4_b4_cache_tests.py`、`g4_b5_eos_branch_tests.py`、`g4_b5_helper_tests.py`、`g4_b5_smoke.py`、`g4_b6_correctness.py`、`g4_b6_fp32_correctness.py`、`g4_b6_tie_diag.py`、`g4_block_logits_semantics.py`、`g4_block_vs_serial.py`、`g4_specdec_index_tests.py`、`g4_part4_*`	draft 正确性与缓存漂移、双模型可行性、cache append/trim/rollback、EOS 分支、主循环不变量、**B6 正确性门（唯一放行判据）**、fp32 档 B6、near-tie 归因、block logits 语义、block vs serial、索引语义哨兵
G5	`g5_8k_smoke_cudnn.py`、`g5_8k_spec_variance_probe.py`、`g5_stability_probe.py`、`g5_cost_breakdown.py`、`g5_k_sweep.py`、`g5_attn_*`、`g5_8k_prefill_finite_revise.py`、`g5_8k_qk32_*`、`g5_8k_residual_history_probe.py`、`g5_8k_divergence_*`、`g5_attn_claims_audit.py`、`g5_attn_cudnn_validate.py`	单次 smoke、跨进程速度双档、计时稳定化、成本分解、k 扫描、注意力后端显存/时间/数值等价、主张↔结果审计、cuDNN 策略验证阶梯、残余分歧缓存历史探针、2×2 定点对照

> 这些是**历史门/历史诊断**：本手册**未逐一重跑**（多数需要 GPU 或完整流程）。
> 它们的状态以各自产物 JSON 与 `docs/` 报告为准；**不要**因为没有重跑就当成通过或失败。

7. 分歧定位诊断（主线，按"问题"组织）

7.1 "引擎 ↔ HF 不一致" 怎么查

问题	工具	实现方法要点
同路径同日程下两侧是否一致?	`xb_probe` + `xb_compare_cpu`	协议 S*；**阶段闸门**只到 C1；克隆不变量断言写进产物；比较工具"未执行 ≠ 相同"
差异出现在哪一层/哪个节点?	`l0_k_replay`（第 0 层）、`align_debug_layer0/align_debug_attn`（历史版）	**节点对照**：`embed → norm → k_pre → cos/sin → k_post`，每个节点记录 dtype/shape/**stride**/sha
差异是"表"还是"旋转运算"?	`l0_k_replay` 的 `subst` 模式	固定**同一份** RoPE 前 K + **同一旋转实现**，只换 cos/sin（A=自研表、B=HF 表），要求 A/B 各自**逐位**等于两侧快照、且差异位置集合相同

问题	工具	实现方法要点
表为什么不同?	`c1_rope_source_diag` 的 `table` 模式	把两侧算式**在各自实际设备上**重算并逐位比较：`inv_freq`（CPU pow vs GPU pow、差异下标）、`outer` vs `matmul`、cos/sin 的 bf16 cast；再与已捕获的实际表核对
对齐后完整 C1 是否一致?	`c1_rope_source_diag` 的 `run` + `compare`	一次完整 prefill 到 8192；只读比较 56 个 K/V 张量与 151936 维 logits

7.2 "AR 与 Spec 不一致" 怎么查

问题	工具	实现方法要点	
真实生成的完整序列是否一致?	**`g5_8k_smoke_cudnn.py`**（`SMOKE_MODE=ar\`	spec`）	AR=**target-only** 单步循环（生产记账顺序：先记录本步 token 再喂下一步）；Spec=`spec_dec.spec_decode_greedy` + `DecisionHooks`；逐阶段落盘；有限性检查在**每次 argmax 之前**；通过判定与"执行结束"分开
首组不一致时下一步做什么?	**`ar_spec_consistency_cpu.py`**	只读三方比较：①配置/权重/精度/输入 ②**初始状态核验**（两臂 + HF C1 参考 + 已对齐诊断运行；K/V 逐张量 sha、logits 同约定 sha、跨工具只用与归约顺序无关的量）③序列比较（token IDs、common prefix、首次分歧 index/绝对位置/两侧 token）④首次分歧对应的**真实轮次**⑤同协议既有运行分组交叉核对	
分歧轮现场怎么拿到?	`g5_8k_smoke_cudnn.py` + `SMOKE_DIVCAP[_POS]`	AR：按 `cache_len` 捕获起点缓存与该行；Spec：经 `DecisionHooks.on_verify_pre/on_verify_post`（**`_verify` 改写缓存之前**的唯一时机）捕获；快照用 `clone_list_cache_full`（**保留全部层内键与长度**，不只 K/V）	
是"执行形状"还是"缓存历史"?	**`f13_i50_cells.py`**（四格）	同前缀 × {AR历史, Spec历史} × {s=1 串行, s=4 分块}；每格**独立完整克隆**并校验不共享存储；**控制格**（真实发生过的组合）必须逐元素复现真实捕获行；`CELLS_REPEAT` 格内重复；`CELLS_LEAK_CHECK` 尾部替换 + **阳性对照**	
数据流有没有错位?	`f13_i50_cells.py` 的 `CELLS_MODE=cpu_prov`	**纯 CPU**：`rows[j]` argmax ↔ **真实生成序列**在该行预测位置上的 token；↔ 运行记录的 `g[j+1]`；AR 行 argmax ↔ 真实 token；rollback 边界 `pre+m` 与 `tgt_len`；"决策槽就是预测目标位置的那一槽"	
两份历史差多少、从哪开始?	同上	逐张量 `torch.equal`（**强于摘要**）+ 每层最早分叉位置（先把 (1,kv,T,d) 的 kv 维与 head_dim 维分别归约，**再**取位置——本项目在这里写错过一次归约维）	

7.3 F06 / F13 定点取证（历史）

`f13_i50_capture.py`（真实路径捕获，`F13_I50_MODE=ar|spec`）、`f13_i50_corrections_cpu.py`（修正与重新标注）、`g5_f06_index25_forensics.py`、`l27_forensics_f06_i25.py`、`layerwise_f06_i25.py`、`lmhead_shape_f06_i25.py`、`mlp_shape_f06_i25.py`、`mlp_tail_f06_i25.py`、`norm_boundary_f06_i25.py`、`substitution_f06_i25.py`、`tail_shape_control_f06_i25.py`、`kv_prefix_cpu_f06_i25.py`、`replay_f06_i25_samehistory.py`、`prefill_sb_replay_f06.py`、`analyze_f06_index25_cpu.py`、`audit_f06_i25_artifacts_cpu.py`、`local_replay_L0_linears.py`、`attn_l27_g012_f06_i25.py`、`attn_l27_tail_ab_f06_i25.py`、`unified_geometry_run.py`。

> 这些工具属于**旧数值谱系（lineage A）**的取证；其**观测**成立，**归因**需按 §1.4.6 重看。
> 使用前先读 `docs/f13_i50_*` 与对应报告章节。

---

8. 治理与记录

工具	作用	实现方法	状态
`verify_family_independence_cpu.py`	**守卫**：候选 prompt family 与调试输入是否互相独立；谁在消费 manifest	①文本扫描 `specdec/engine` + `scripts` 找 manifest 消费方，要求集合与 `OPT_IN_CONSUMERS` **完全一致**；②**经验证据**：扫 `output/g5/attn/*.json` 的 `input.source_kind/family`；③授权范围校验 + **每个登记消费方一条范围检查**（该工具只允许消费某个 family）	**25 PASS / 0 FAIL**；新增消费方一律"显式登记 + 范围收窄"，**从未**因报错而放宽检查
`family16_summary.py`	16 格清单的纯 CPU 汇总	只读 run JSON 与索引	历史
`results_schema.py`（+ `g5_schema_selftest_cpu.py`）	结果 schema、判定分类器（含 `classify_bf16_compare` 的四条件门槛）、MAD/IQR、run 元数据	纯函数 + 162 项自测	已验证
`g5_driver.py`、`status_fields.py`	系统快照/GPU 消费者；**五类状态**（诊断/正确性/验收/性能/环境）分开记录	只读采集 + 汇总函数	库
`gpu_pstate_preflight.py`	GPU active-load P-state 资格预检	采样 sm/mem 中位数判定	库（正式计时前必过）

9. 方法论：这些工具共同的实现方法

9.1 复现门（先复现，再归因）

任何定位前先证明"我能重放出现象"：重放/重跑的结果必须与**既有快照逐位相同**（`k_post`、`v`、完整 64-token 序列与 `gen_hash`）。未过门 ⇒ 落盘停止，**不**用后续差异讨论正确性。

本项目的 C1 定位、L0 定位、AR/Spec 定位都走这条门。

9.2 控制格 vs 干预格（不为没发生过的组合编造参考）

四格里只有 **H_AR-S**（AR 历史 × 串行）与 **H_SP-B**（Spec 历史 × 分块）在真实运行里**发生过** ⇒ 只要求这两格逐元素复现真实捕获行；另外两格是**干预**，**不设参考、不编造参考**。

9.3 摘要约定（很容量错的地方）

- `stable_sha`：`sha256(t.float().cpu().contiguous().numpy().tobytes())[:16]` ⇒ **跨进程可比。**
- **只在同一工具、同一约定内用 sha 判等。**例：smoke 工具哈希的是整块 `(1,2048,V)` 输出，而 `c1rope` 哈希的是末位 `(V,)` 行 ⇒ 两者 sha **本来就不该相等**，直接比会得出错误结论（本项目踩过）。
- **跨设备**：GPU 上求的 `sum` 与 CPU 上重算的 `sum` 会因**归约顺序**差 1 个 fp32 ULP ⇒ 只用 `argmax`/`max`/`margin` 这类与归约顺序无关的量做跨工具对位。
- **逐张量 `torch.equal` 强于摘要**；能在同一进程里比就比逐位。

9.4 泄漏检查必须带阳性对照

`CELLS_LEAK_CHECK`：把**决策行之后**的尾槽 token 换掉 ⇒ 决策行必须**逐位不变**（无泄漏）；**同时**要求被替换的尾槽**自身的行确实改变**（阳性对照）。没有阳性对照的"没变化"是**空证据。**

9.5 四格与归因规则

`同一历史换形态 ⇒ 支持"执行形状影响"`；`同一形态换历史 ⇒ 支持"缓存历史影响"`；两者并存或依赖历史时**如实报告交互**，不强行选唯一原因。

margin 大小不能单独判定逻辑错误或数值误差（本项目的 `classify_bf16_compare` 还额外要求 `fp32_margin`，缺证据一律 `NEEDS_INSPECTION` ⇒ **缺证据不得豁免**）。

9.6 快照完整性

缓存快照**必须**用 `cache_clone.clone_list_cache_full`（`copy.deepcopy`）：保留**全部层内键**（张量 + 非张量元数据）与长度信息。**只复制 K/V 就是 bug**（`b0db582` 修掉的正是这一类：HF `DynamicLayer.is_initialized` 丢了 ⇒ 首次 `update()` 清空历史）。

9.7 lineage 纪律

跨谱系（A/B）**不混比**；新产物必须写明 `target_rope` / `rope_inv_freq_source`；历史产物**原样保留**。

9.8 失败保留与不覆盖

- 新产物全部带 `run_id`/时间戳；**绝不覆盖**历史文件；比较类工具输出名带**写入时间戳**。
- 失败记录（崩溃 JSON/日志、注入故障自测、被撤回的结论）**一律保留**，原文加标记而不删除。
- 每次改动都在产物里写清"来源/口径/强度"，让后来者能判断"这条数字能不能用"。

9.9 进程与环境模型

- **每次命令行调用是新进程**：环境变量不保留 ⇒ 变量与命令写在同一行。
- 诊断与真实运行**分进程**、**GPU 串行**（绝不双模型以外并发）；每个真实运行独立 `commit_guard` + P-state 预检。
- 计时只作诊断，**禁止**进入任何 speedup 表；`allocated ≠ 物理占用`，`reserved` 超物理容量**不单独**证明分页。

9.10 陷阱清单（本项目实际踩过的，按"教训"记）

#	陷阱	后果	现在的做法
1	包装函数的**形参名**与生产函数不一致（`theta_` vs `theta=` 关键字调用）	崩溃	形参名与原函数**逐字一致**；CPU 自测会先抓到
2	只复制 K/V 的克隆	静默丢历史 ⇒ 整轮对照无效	统一 `clone_list_cache_full` + 不变量断言
3	**不同约定**的 sha 直接比较	得出错误的"未通过"结论	只在同约定内比 sha；跨工具用与归约顺序无关的量
4	**跨设备** `sum` 判等（GPU vs CPU 归约顺序）	差 1 ULP 被当成差异	剔除该判等并写明理由
5	归约维写错（把 head_dim 当位置）	得出错误的"prefill 也不同"	显式写清每个 `dim` 的含义；与已知事实矛盾时先怀疑代码
6	先 `_run` 再记录 token / 漏 `inference_mode`	整条序列错位一位 / 8K OOM	恢复生产记账顺序与 `inference_mode`（**不是**放宽条件）
7	保存路径 typo（`o` vs `out`）	**已算完的一次 prefill 数值全丢**	落盘前先做"字段自检"；分阶段落盘
8	固定输出文件名	覆盖掉一条派生记录	输出名带写入时间戳 + 已存在则加序号
9	环境变量没跟命令写在一行	跑成默认配置，浪费一次 GPU 运行	一行写完；产物里核对 `config`
10	用手写 `theta`/`head_dim` 而不是读 config	数值口径错	一律从 config / 模型实例读，并在产物里记录

10. 命令速查

```
# 0) 全量 CPU 自测（§3 的循环）
# 1) 治理守卫
.\.venv\Scripts\python.exe -X utf8 -m specdec.engine.verify_family_independence_cpu

# 2) 克隆不变量 / 作用域 / 四格 / 跨实现自测（内置）
.\.venv\Scripts\python.exe -X utf8 -m specdec.engine.cache_clone
.\.venv\Scripts\python.exe -X utf8 -m specdec.engine.target_rope_scope
.\.venv\Scripts\python.exe -X utf8 -m specdec.engine.f13_i50_cells      # 需 CELLS_SELFTEST=1
.\.venv\Scripts\python.exe -X utf8 -m specdec.engine.xb_probe          # 需 XB_SELFTEST=1

# 3) RoPE 对齐诊断（gate 不加载模型）
$env:L0D_MODE="gate";  .\.venv\Scripts\python.exe -X utf8 -m specdec.engine.c1_rope_source_diag
$env:L0D_MODE="run";   .\.venv\Scripts\python.exe -X utf8 -m specdec.engine.c1_rope_source_diag
$env:L0D_MODE="compare"; .\.venv\Scripts\python.exe -X utf8 -m specdec.engine.c1_rope_source_diag

# 4) 真实 AR / Spec 一次运行（默认关闭的分歧捕获）
$env:SMOKE_MODE="ar"; $env:SMOKE_ARM="eager"; $env:SMOKE_FAMILY="F13"; $env:SMOKE_CTX="8192"; `
$env:SMOKE_CHUNK="2048"; $env:SMOKE_K="4"; $env:SMOKE_MAXNEW="64"; $env:SMOKE_KV="base_bf16"; `
$env:SMOKE_TARGET_ROPE="cpu_fp32"; $env:SMOKE_BF16_RED_REDUCTION="1"; $env:SMOKE_PREFILL_DIGEST="1"; `
$env:SMOKE_STRUCT_CHECK="1"; $env:SMOKE_DIVCAP="1"; $env:SMOKE_DIVCAP_POS="8239"; `
.\.venv\Scripts\python.exe -X utf8 -m specdec.engine.g5_8k_smoke_cudnn

# 5) 四格（历史来源 × 执行方式）
$env:CELLS_TAG="divasc_20261001"; $env:CELLS_PRE="8237"; $env:CELLS_TARGET="8239"; `
$env:CELLS_K="4"; $env:CELLS_FEED="5880,389"; $env:CELLS_CANDS="5880,389,5470,220"; `
$env:CELLS_WATCH="6702,6652,5470"; $env:CELLS_REPEAT="2"; $env:CELLS_LEAK_CHECK="1"; `
$env:CELLS_TARGET_ROPE="cpu_fp32"; $env:CELLS_AR_ROW_KEY="row_target"; `
$env:CELLS_CAP_AR="…\divcap_ar_….pt"; $env:CELLS_CAP_SP="…\divcap_spec_….pt"; `
.\.venv\Scripts\python.exe -X utf8 -m specdec.engine.f13_i50_cells
# 只读数据流核对（不加载模型）
$env:CELLS_MODE="cpu_prov"; $env:CELLS_REF_AR="…smoke8k_eager_….json"; $env:CELLS_REF_SP="…"; `
.\.venv\Scripts\python.exe -X utf8 -m specdec.engine.f13_i50_cells

# 6) AR/Spec 一致性只读比较
.\.venv\Scripts\python.exe -X utf8 -m specdec.engine.ar_spec_consistency_cpu

# 7) 本手册 PDF
.\.venv\Scripts\python.exe -X utf8 scripts\md_to_pdf.py docs\test_code_handbook_2026-10-01.md --inventory
```

11. 附录 B：文档与产物索引（本阶段）

类型	路径
冻结提案	`docs/proposal_freeze_2026-10-01.md`
缓存克隆勘误 + 过度推断更正	`docs/xb_clone_bug_errata_2026-10-01.md`（§11 勘误之二）
L0 K 差异定位	`docs/l0_k_localization_2026-10-01.md`
RoPE 频率对齐（C1 逐位一致）	`docs/c1_rope_source_alignment_2026-10-01.md`
对齐后 AR/Spec 真实生成一致性	`docs/ar_spec_aligned_rope_2026-10-01.md`
index 47 分歧定位	`docs/divasc_index47_localization_2026-10-01.md`
阶段报告 / 数据台账	`docs/cudnn_prefill_stage_report.md`（§9.43–§9.47）、`docs/data_ledger.md`（§4.58–§4.62）
运行产物	`output/g5/attn/`（`smoke8k_*`、`divcap_*`、`divasc_*`、`c1rope_*`、`l0k_*`、`l0_replay_*`、`hfsb_*`、`xb_*`、`f13i50_*`、`ucg_*`）

> **提醒**：`output/` 被 `.gitignore` 忽略 ⇒ 产物不进 git；需要长期保存的关键数字请写进 `docs/` 的报告与台账。

附录 A：`specdec/engine` 全量代码清单（自动扫描）

共 **139** 个 `.py`；其中带 `__main__` 的 **121** 个、含自测入口的 **8** 个。本表由 `scripts/md_to_pdf.py --inventory` 在生成 PDF 时**实时扫描**，因此新增文件会自动出现在下一次生成的 PDF 里。

文件	行数	作用（模块 docstring 首行）	入口环境变量	自测
`__init__.py`	1	specdec.engine — from-scratch LLM engine (OURS).	—	
`align_cache_diag.py`	60	where do full-forward and cached-decode diverge?	—	
`align_cache_real.py`	62	cache==full check on REAL Qwen weights at several depths.	—	
`align_debug.py`	77	per-layer diff: where do our logits diverge from HF?	—	
`align_debug_attn.py`	98	bisect INSIDE attention layer 0.	—	
`align_debug_layer0.py`	92	bisect INSIDE layer 0: norm? attention? mlp?	—	
`align_diverge_diag.py`	72	at the first divergence, compare top tokens from:	—	
`align_generate.py`	97	L2 generation alignment gate (near-tie aware).	—	
`align_ours.py`	118	L1 alignment gate for the student's OWN engine (model_self.py).	—	
`alignment.py`	71	engine.alignment — A4 gate: our from-scratch forward vs HuggingFace Qwen2.	—	
`analyze_f06_index25_cpu.py`	167	F06/index25 取证结果的**纯 CPU** 汇总（不加载模型）。	—	
`ar_spec_consistency_cpu.py`	394	**只读**比较：target RoPE 对齐后的真实 AR 与 Spec 完整生成（纯 CPU）。	ARSPEC_AR_RUN, ARSPEC_SPEC_RUN	

文件	行数	作用（模块 docstring 首行）	入口环境变量	自测	
`attn_l27_core.py`	114	L27 attention 核心的**功能式**实现（生产表达式逐字对应）。	—		
`attn_l27_g012_f06_i25.py`	402	L27 attention 最小对照实验：**G0 / G1 / G2**（F06/index25）。	—		
`attn_l27_g012_selftest_cpu.py`	161	G0/G1/G2 运行前的 CPU 测试（不加载模型、不用 GPU）。	—		
`attn_l27_tail_ab_f06_i25.py`	294	L27 attention 行数差异的**末端传播验证**（A/B，有界局部干预）。	—		
`attn_l27_tail_ab_selftest_cpu.py`	115	A/B 末端传播实验的**运行前 CPU 自测**。	—		
`attn_phase_selftest_cpu.py`	317	prefill-only cuDNN attention 分支的**纯 CPU** 自测。	—		
`attn_tail_common.py`	81	A/B 末端传播实验的**共享助手**（CPU 可测；不含任何模型调用）。	—		
`audit_f06_i25_artifacts_cpu.py`	262	F06/index25 产物覆盖审计（**纯 CPU，只读**）。	—		
`bench_basic.py`	83	first performance numbers (L2 milestone: decode tokens/s).	—		
`c1_rope_source_diag.py`	432	**只对齐 RoPE 频率来源**后的 C1 诊断（`L0D_MODE=gate	run	compare`）。	C1R_PRIOR_ATTEMPT, L0D_MODE
`cache_clone.py`	349	HF `DynamicCache` 的**可靠克隆**与 8 项 CPU 硬测试（可 `python -m` 运行自测）。	—	✓	

文件	行数	作用（模块 docstring 首行）	入口环境变量	自测	
`cpu_nll_sanity.py`	88	HF sanity audit: 引擎 vs HF 在 CPU fp32 的 NLL/argmax（不碰 GPU）	—		
`decision_rows.py`	127	决策行对齐的纯逻辑（CPU 可测）。	—		
`decision_rows_selftest_cpu.py`	140	决策行对齐逻辑的 CPU 自测（先自测，再捕获）。	—		
`diag_cache_cmp.py`	320	KV cache 的**真正独立**快照与**可检测**比较（纯函数，可 CPU 自测）。	—	✓	
`diag_checks.py`	209	本轮**修复后的**诊断校验原语（纯函数/纯断言，可 CPU 自测）。	—	✓	
`diag_fields.py`	127	诊断**字段语义**的单一来源（纯函数，可在 CPU 上测，不加载大模型）。	—		
`diag_fields_selftest_cpu.py`	119	`diag_fields` 的轻量回归（**不加载大模型**）。	—		
`diag_qk_scope.py`	168	把 **QK FP32 诊断开关限定到指定模型实例**，并给出**干预范围**的静态/动态证明。	—		
`dispatch_judge.py`	96	阶段分流（attention backend）的**按后端**判定（纯函数，CPU 可测）。	—		
`dispatch_judge_selftest_cpu.py`	146	按后端阶段分流判定的 CPU 自测。	—		
`f13_i50_capture.py`	383	F13 index50 首次分歧的**真实路径捕获**（`F13_I50_MODE=ar	spec`）。	F13_I50_MODE	
`f13_i50_cells.py`	693	**同前缀四格局部对照**（历史来源 × 执行方式），不自由生成。	CELLS_ARM, CELLS_AR_ROW_KEY, CELLS_CANDS, CELLS_CAP_AR, CELLS_CAP_SP, CELLS_CHUNK, CELLS_FEED, CELLS_K, CELLS_LEAK_CHECK, CELLS_MODE, CELLS_PRE, CELLS_PROBE_A, CELLS_PROBE_B, CELLS_REF_AR, CELLS_REF_SP, CELLS_REPEAT, CELLS_SELFTEST, CELLS_TAG, CELLS_TARGET, CELLS_TARGET_ROPE, CELLS_WATCH	✓	

文件	行数	作用（模块 docstring 首行）	入口环境变量	自测
`f13_i50_corrections_cpu.py`	150	F13/index50 诊断记录的**修正与重新标注**（纯 CPU，不新增 GPU 运行）。	—	
`family16_summary.py`	199	16 格清单的**汇总**（纯 CPU；只读 run JSON 与索引）。	REPEAT_ON_DIVERGENCE, SUMMARY_INDEX	
`family_input.py`	190	G1：按 **manifest family** 取 8K 输入（显式启用，默认路径不变）。	SMOKE_FAMILY, SMOKE_MANIFEST	
`fp8_kv_group_probe.py`	247	FP8 KV 配置诊断探针（D 语义保留; 不覆盖正式脚本）	—	
`fp8_kv_v1_roundtrip.py`	100	G3 验证 V1 (B 版): groupwise(32) FP8 KV 往返单测（CUDA）	—	
`fp8_kv_v2_short_window.py`	148	G3 验证 V2: fp8-vs-fp16 短窗行为（CUDA, 需引擎已实现 set_kv_format）	—	
`fp8_kv_v3_chunked_ablation.py`	159	G3 验证 V3: chunked fp8 消融（CUDA, 需引擎已实现 set_kv_format）	—	
`fp8_kv_v3_localize.py`	224	V3 分叉定位(修正版): prefill 与 decode 分离, 纯 prefill KV 比较	—	
`fp8_kv_v4_dppl.py`	214	G3 正式质量门 V4: base(BF16 KV) vs Asymmetric V-only FP8(K BF16 + V FP8 gs128)	—	
`fp8_kv_v4_probe.py`	108	V4 差分探针: 判定超高 PPL 是“instruct 模型裸文本退化”还是“引擎 NLL 路径 bug”	—	
`fp8_scale_diag.py`	82	量化误差归因诊断(零引擎改动): 决定B(分组 scale)是否值得	—	
`fp8_vonly_8k_operational.py`	132	8K operational: base(BF16 KV) vs Asymmetric V-only FP8	—	

文件	行数	作用（模块 docstring 首行）	入口环境变量	自测
`fp8_vonly_v1_roundtrip.py`	155	正式 Asymmetric V-only FP8 (gs=128) 直接结构/往返测试	—	
`g1_target_15b.py`	120	G1: Qwen2.5-1.5B-Instruct target correctness gate.	—	
`g2_feasibility.py`	106	G2 minimal feasibility: 8K × 1.5B chunked prefill on GPU.	—	
`g2_operational.py`	228	正式 G2 门: 8K Operational Feasibility（v7.1 §7 + P11）	—	
`g2_peak_probe.py`	95	G2 峰值显存方差探针（定性 6.83 vs 7.62 GiB 跳变）	—	
`g4_0_draft_correctness.py`	172	G4-0: 0.5B draft correctness gate（engine OURS, BF16 CUDA）	—	
`g4_0_drift_diag.py`	242	G4-0 cache 漂移归因诊断（0.5B draft, BF16 CUDA; 另含 FP32 控制）	—	
`g4_b3_dual_model_feasibility.py`	209	B3: 双模型(1.5B target + 0.5B draft) 8GB 可行性（BF16 CUDA）	—	
`g4_b4_cache_tests.py`	216	B4: cache append/trim/rollback 测试（对用户实现的 spec_cache 契约）	—	
`g4_b5_eos_branch_tests.py`	176	EOS 分支的**纯逻辑**回归测试（无需模型/GPU/网络, CPU 毫秒级）	—	
`g4_b5_helper_tests.py`	172	B5 分段自测: 先验证 spec_dec 的"零件", 再写主循环	—	
`g4_b5_smoke.py`	89	B5: greedy SpecDec 主循环「结构不变量」自检（不判正确性）	—	

文件	行数	作用（模块 docstring 首行）	入口环境变量	自测
g4_b6_correctness.py	146	B6: SpecDec exact-token-id correctness gate（唯一放行证据）	—	
g4_b6_fp32_correctness.py	194	B6-fp32：高精度档下的 SpecDec **实现**正确性门（预登记）	B6F_LIMIT	
g4_b6_tie_diag.py	237	B6 唯一失败 case 的 near-tie 归因（base_bf16 / natural-2）	—	
g4_block_logits_semantics.py	134	用**真实模型**验证 block forward 的 logits 语义（CPU fp32 控制）。	P4S_K, P4S_S, P4S_TOL	
g4_block_vs_serial.py	166	target block-forward vs serial cached-forward 一致性诊断	—	
g4_part4_8k_isolated_probe.py	402	8K 四条件的**独立进程**复验 + 离线报表。	—	
g4_part4_mismatch_diag.py	339	定位历史 Part 4 中 SpecDec != AR 的三个条件（确定性诊断）。	P4D_CHUNK, P4D_CTX, P4D_FP32, P4D_MAXNEW, P4D_NOISE	
g4_part4_pilot_measure.py	916	G4 Part 4 测量 + 报表（**只出数，不作加速声明**）	P4_AR_RESIDENCY, P4_CHUNK, P4_CTX, P4_K, P4_KV, P4_MAXNEW, P4_MODE, P4_ONLY, P4_ORDER, P4_REL_SPREAD_LIMIT, P4_REPS, P4_REVERSE, P4_RUN_ID, P4_TEXT, P4_WARM, PYTORCH_CUDA_ALLOC_CONF	
g4_part4_selftest_cpu.py	310	Part 4 测量工具链的 **CPU 自测**（不加载模型、不碰 GPU）。	—	
g4_specdec_index_tests.py	251	SpecDec **logits 索引语义**的确定性哨兵测试（CPU，无模型）。	—	
g5_8k_divergence_2x2.py	625	用**真实两条路径的真实 cache 历史**做 2×2 定点对照（TOOL_VERSION=2）。	DIAG_CHUNK, DIAG_CTX, DIAG_K, DIAG_KV, DIAG_MAX_NEW, DIAG_MODE, DIAG_PREFIX_TOKENS, DIAG_TAG	
g5_8k_divergence_2x2_selftest_cpu.py	114	2×2 诊断工具**纯逻辑**的 CPU 自测（不加载模型）。	—	

文件	行数	作用（模块 docstring 首行）	入口环境变量	自测
`g5_8k_divergence_diag.py`	264	**定点**诊断 8K 同后端 AR/SpecDec 的首个分歧（阶段 1：同 cache、不同形状）。	DIAG_CHUNK, DIAG_CTX, DIAG_FP32_ATTN, DIAG_KV, DIAG_PREFIX_TOKENS	
`g5_8k_prefill_finite_revise.py`	170	从**已有 JSON 记录**离线复核 `prefill_logits_finite`（纯 CPU）。	REVISE_ALL, REVISE_TAGS	
`g5_8k_qk32_analysis.py`	366	读 8 份结果，产出"控制组 vs 干预组"表与分歧证据（**纯 CPU**）。	—	
`g5_8k_qk32_interention.py`	401	QK FP32 最小精度干预测试（控制组 vs 干预组，完整 64-token 生成）。	QK_ARM, QK_CHUNK, QK_CTX, QK_GROUP, QK_K, QK_KV, QK_MAXNEW, QK_MODE, QK_TAG	
`g5_8k_residual_history_probe.py`	501	base_bf16 干预组**残余分歧**的缓存历史单变量探针。	RP_ARM, RP_CACHE_LEN_BEFORE, RP_CHUNK, RP_CTX, RP_DIV_INDEX, RP_K, RP_KV, RP_MAXNEW, RP_MODE, RP_REF_GROUP, RP_REF_TAG, RP_TAG	
`g5_8k_smoke_cudnn.py`	827	**8K 单次 smoke**（条件性工具；只回答"能不能跑、峰值多少"）。	REPEAT_ON_DIVERGENCE, SMOKE_ALLOW_UNQUALIFIED, SMOKE_ARM, SMOKE_BF16_RED_REDUCTION, SMOKE_CHUNK, SMOKE_CTX, SMOKE_DIVCAP, SMOKE_DIVCAP_POS, SMOKE_DRY_RUN, SMOKE_K, SMOKE_KV, SMOKE_MAXNEW, SMOKE_MODE, SMOKE_PREFILL_DIGEST, SMOKE_SELFTEST, SMOKE_STRUCT_CHECK, SMOKE_TARGET_ROPE	✓
`g5_8k_spec_variance_probe.py`	1063	**8K SpecDec 跨进程速度双档专项定位**（诊断工具，不是吞吐测量）。	P5V_ALLOW_UNQUALIFIED, P5V_BIMODAL_RATIO, P5V_CHILD, P5V_CHUNK, P5V_COOLDOWN, P5V_CTX, P5V_DRY_RUN, P5V_INDEX, P5V_K, P5V_KV, P5V_MAXNEW, P5V_MIN_COMMIT_GIB, P5V_PROCS, P5V_PROF_REPS, P5V_REPS, P5V_ROLE, P5V_ROLES, P5V_RUN_ID, P5V_WARM, P5V_WDDM, PYTORCH_CUDA_ALLOC_CONF	
`g5_attn_backend_probe.py`	274	attention 后端的**显存 / 时间 / 数值等价性**微基准。	—	
`g5_attn_claims_audit.py`	321	**报告主张 ↔ 原始结果** 的机器可核对审计（不需要 GPU）。	—	
`g5_attn_cudnn_validate.py`	625	`cudnn_prefill` 策略的 **GPU 验证阶梯**（任务验证顺序第 2、3 步）。	ACV_ALLOW_UNQUALIFIED, ACV_CHUNK, ACV_CTXS, ACV_CTXS_2KV, ACV_DRY_RUN, ACV_FUSED_MAX_MIB, ACV_FUSED_RATIO_MAX, ACV_K, ACV_KVS, ACV_MAXNEW, ACV_REPS, ACV_STAGE, ACV_TOL_ABS, ACV_TOL_REL, HF_HUB_OFFLINE, PYTORCH_CUDA_ALLOC_CONF	
`g5_attn_nll_quality.py`	374	attention 后端的**诊断性质量检查**（mean NLL / PPL，GPU）。	NLL_ALLOW_UNQUALIFIED, NLL_HF, NLL_KV, NLL_LOWMEM_LOAD, NLL_N, NLL_W	

文件	行数	作用（模块 docstring 首行）	入口环境变量	自测
`g5_attn_precision_forensics.py`	219	定位"eager vs 融合"数值差异来源的**专项取证**（GPU，不加载权重）。	FORENSICS_DEVICE	
`g5_attn_real_tensor_forensics.py`	692	**真实模型张量**上的逐层/逐项精度定位（GPU，只加载 target）。	RTF_CHUNK_B, RTF_CTX_A, RTF_CTX_B, RTF_KV, RTF_LAYERS, RTF_LOWMEM_LOAD, RTF_STAGE	
`g5_chunk_fit_sweep.py`	212	**chunk 分档拟合扫描**：在不改引擎的前提下，找出 8K SpecDec 能跑通的最大 chunk。	—	
`g5_cost_breakdown.py`	301	Task C：SpecDec / AR 的阶段成本分解（**profiling，不是正式计时**）。	P5C_CHUNK, P5C_CTX, P5C_INCLUDE_DUAL_AR_CONTROL, P5C_K, P5C_KV, P5C_MAXNEW, P5C_REPS, P5_PROFILE	
`g5_driver.py`	317	G5 测量驱动层的共用工具（**不加载模型、不含 GPU 代码**，CPU 可测）。	HF_HUB_OFFLINE, P5_MIN_ACTIVE_MEM_CLOCK_MHZ, P5_MIN_ACTIVE_SM_CLOCK_MHZ	
`g5_f06_index25_forensics.py`	434	定位 F06 / base_bf16 首次 AR–Spec 分歧（index 25）的具体来源。	PROBE_ARM, PROBE_CHUNK, PROBE_CTX, PROBE_DIV_INDEX, PROBE_EXPECT_AR, PROBE_EXPECT_SPEC, PROBE_FAMILY, PROBE_K, PROBE_KV, PROBE_MAXNEW, PROBE_MODE, PROBE_TAG	
`g5_k_sweep.py`	157	Task D：有限 k 扫描（**诊断实验，不修改正式协议**）。	P5KS_CHUNK, P5KS_COOLDOWN, P5KS_CTX, P5KS_KS, P5KS_MAXNEW, P5KS_REPS, P5KS_WARM	
`g5_schema_selftest_cpu.py`	894	G5 结果 schema 的纯 CPU 自测（不加载模型、不碰 GPU）。	P5_MODE	
`g5_stability_probe.py`	456	G5 计时稳定化探针（Task B）+ AR 基线口径分离 + P-state 门禁。	P5_ALLOW_UNSTABLE_CLOCK, P5_BLOCKS, P5_CHUNK, P5_COOLDOWN, P5_INCLUDE_DUAL_AR_CONTROL, P5_MAXNEW, P5_REPS, P5_REQUIRE_AC, P5_SEED, P5_WARM	
`gpu_bf16_align_probe.py`	121	bf16 对齐探针: OURS vs HF（CUDA bf16, 中等长度）	—	
`gpu_fp16_align_probe.py`	125	决定性对齐探针: OUR GPU fp16 vs HF GPU fp16 的 logits	—	
`gpu_pstate_preflight.py`	189	GPU **active-load P-state 资格预检**（独立、可复用、非正式计时）。	P5_MIN_ACTIVE_MEM_CLOCK_MHZ, P5_MIN_ACTIVE_SM_CLOCK_MHZ, P5_PSTATE_MATRIX, P5_PSTATE_PREFLIGHT_SECONDS, P5_PSTATE_SAMPLE_INTERVAL_S	

文件	行数	作用（模块 docstring 首行）	入口环境变量	自测
`hf_serial_block_probe.py`	554	**HF 单步 / 块计算对照**（外部实现，独立诊断入口）。	HFSB_CHUNK, HFSB_CTX, HFSB_FAMILY, HFSB_MODE, HFSB_PRE, HFSB_RED_REDUCTION, HFSB_TINY, HFSB_TOKENS	✓
`kv_prefix_cpu_f06_i25.py`	152	用**已有产物**做 L27 K/V 数值核验（**纯 CPU，只读**）。	—	
`l0_k_diff_cpu.py`	215	**零 GPU**：从既有两份 C1 快照里取出第 0 层 K/V 的全部差异清单（只读，不改任何历史产物）。	—	
`l0_k_replay.py`	604	**第 0 层 K 生成的最小重放**：节点对照 + 复现门 + 条件式 RoPE 表替换。	L0_HF_DUMP, L0_MODE, L0_OURS_DUMP, L0_PRIOR_ATTEMPT, USE_HUB_KERNELS	
`l27_forensics_f06_i25.py`	267	定位 L27 内部的对齐行差异（只读 hook，不改 forward）。	—	
`layerwise_f06_i25.py`	318	F06/base_bf16/index25 的**逐层**对照：块路径(s=4) vs 串行路径(s=1)。	—	
`lmhead_shape_f06_i25.py`	246	隔离**输出端**：隐藏向量来源 × LM head 处理形状（F06/index25）。	—	
`local_replay_l0_linears.py`	146	§六 局部同输入实验：把 L0.q_proj / L0.o_proj 单独拿出来重放。	—	
`lowmem_load.py`	82	与 `g4_b4_cache_tests.load_ours` **权重逐位相同**、但 CPU 峰值约减半的加载器。	—	
`mlp_shape_f06_i25.py`	256	隔离 L27 MLP 的**输入来源效应**与**调用形状效应**（F06/index25）。	—	
`mlp_tail_f06_i25.py`	339	修正版：严格三维 MLP 四格 + 有界尾部重组（F06/index25）。	—	

文件	行数	作用（模块 docstring 首行）	入口环境变量	自测
`model.py`	182	engine.model — from-scratch Qwen2-family forward pass (L1, no KV cache yet).	—	
`model_self.py`	580	—	—	
`norm_boundary_f06_i25.py`	292	核验**最终 RMSNorm 边界**：差异在上游，还是 Norm 调用形状也有贡献。	—	
`prefill_finite_selftest_cpu.py`	157	`prefill_finiteness` 契约 v2 的 CPU 回归自测（不加载模型）。	—	
`prefill_sb_replay_f06.py`	336	初始状态 × 后续计算路径的 2×2 对照（F06/index25，base_bf16）。	PREFSB_ARM	
`probe_gpu.py`	119	decode-baseline sanity + 8K VRAM probe (GPU fp16).	—	
`qk_scope_selftest_cpu.py`	183	QK FP32 干预的**范围与生效性** CPU 自测（不加载大模型）。	—	
`replay_f06_i25_samehistory.py`	163	同一份 cache 历史下，**块验证行 vs 单步路径**对照（§4 的判决实验）。	—	
`sampling.py`	28	engine.sampling — sampling utilities for decode (OURS). L1 uses greedy only.	—	
`sanity_hf_decode.py`	35	quick reference: transformers eager decode tok/s on GPU.	—	
`sb_align.py`	230	S/B 对照的**对齐与可比性**纯逻辑（CPU 可测；不含模型调用）。	—	
`sb_align_selftest_cpu.py`	197	S/B 对照对齐逻辑的 CPU 自测（先自测，再运行 GPU）。	—	

文件	行数	作用（模块 docstring 首行）	入口环境变量	自测
`smoke_checks.py`	395	8K smoke 的**检查注册表与通过判定**（纯函数；可在 CPU 上做轻量回归）。	—	
`smoke_checks_selftest_cpu.py`	155	8K smoke **检查逻辑**的轻量回归（**不加载任何模型**）。	—	
`smoke_hooks.py`	176	诊断用 hook：**阶段标注** + **决策点有限性检查**（含 `_propose` 中间步）。	—	
`smoke_midstep_selftest_cpu.py`	202	**`_propose` 中间步**有限性检查的轻量集成测试（**不加载大模型**）。	—	
`spec_cache.py`	65	—	—	
`spec_dec.py`	116	—	—	
`spec_verify_index_selftest_cpu.py`	149	`_verify` 的**索引/位置/mask/cache 语义**哨兵（CPU，tiny 模型）。	—	
`status_fields.py`	267	**五种状态分开记录**（纯函数，可 CPU 自测）。	—	
`substitution_f06_i25.py`	267	四臂中间值替换诊断：L0 的 q_proj / o_proj 差异是否影响 index25 选词。	—	
`tail_shape_control_f06_i25.py`	198	为什么尾部重建与真实块行差 0.0625？（只读复用保存张量）	—	
`target_rope_scope.py`	298	把「CPU-FP32 的 RoPE 频率来源」**只**绑定到指定模型实例（诊断用）。	—	✓
`test_cache_equiv.py`	45	L2 correctness gate: cached decode == full forward.	—	
`test_chunked_fp32.py`	99	isolate the 0.55 cache gap: is it fp16 rounding or a real bug?	—	

文件	行数	作用（模块 docstring 首行）	入口环境变量	自测	
`test_chunked_prefill.py`	111	chunked prefill == single-shot prefill (correctness gate).	—		
`unified_geometry.py`	153	**统一执行几何（UCG）**最小原型的共享逻辑。	—		
`unified_geometry_run.py`	234	UCG 原型运行（`UG_MODE=ar	spec， UG_FAMILY=F13	F06`）。	UG_CHUNK, UG_CTX, UG_FAMILY, UG_K, UG_MAXNEW, UG_MODE
`unified_geometry_selftest_cpu.py`	152	UCG 原型的 CPU / 短输入测试（**GPU 之前必须过**）。	—		
`verify_family_independence_cpu.py`	390	验证候选 prompt family 与调试输入**互相独立**（纯 CPU，不用 GPU）。	—		
`verify_readiness_selftest_cpu.py`	281	G1–G5 与五类状态的 CPU 回归自测（不加载模型、不碰 CUDA）。	SMOKE_FAMILY		
`xb_compare_cpu.py`	230	跨实现（OURS vs HF）同路径对照的**只读**比较（纯 CPU）。	—		
`xb_hfsb_row_recompute_cpu.py`	152	**只读重算**：用既有产物做「同协议跨实现行对照」（纯 CPU，不跑 GPU）。	—		
`xb_probe.py`	535	**自研引擎 vs HF 的同路径对齐探针**（`XB_IMPL=hf	ours，另有 XB_SELFTEST=1`）。	XB_CHUNK, XB_CTX, XB_FAMILY, XB_IMPL, XB_PRE, XB_SELFTEST, XB_STAGE, XB_TOKENS	✓